

# Comparativo de Complexidade Computacional entre Implementações Vetorizada e Não-Vetorizada da Regressão Logística para Classificação Binária

Discentes: Eduardo Lucena de Moura, Francisco Gutierrez Aires De Sousa, Gabriel Siqueira dos Santos

Docente: Adam Dreyton Ferreira dos Santos

## Sumário

1. Resumo
2. Introdução
3. Fundamentação Teórica
4. Metodologia
5. Resultados e Discussão
6. Conclusão
7. Referências Bibliográficas

## 1. Resumo

Esse estudo serve para demonstrar o impacto que a vetorização causa no desempenho computacional em Algoritmo especificamente de Regressão Logística. Com a utilização do conjunto de dados “Heart Disease Dataset” do Kaggle, foram implementadas duas versões do algoritmo: uma vetorizada e não vetorizada. A metodologia inclui o pré-processamento dos dados e as próprias implementações, utilizando-se de treinamento dos modelos com gradiente descendente ao longo de 1000 iterações e 5 repetições para cada tipo de implementação, além da medição de tempo e speedup delas. Os resultados demonstram a diferença expressiva: speedup de 409,99x, demonstrando que a eliminação quase completa dos laços “for” de repetição por meio de rotinas otimizadas retiradas da biblioteca Numpy que explora instruções SIMD e operações matriciais do BLAS, diminuindo drasticamente o tempo de busca do modelo sem comprometer acurácia ou custo. Concluindo, a vetorização é um requisito fundamental para a eficiência de algoritmos que trabalham nesse cenário de grande volumes de dados.

**Palavras-chave:** Regressão Logística, Vetorização, Gradiente Descendente.

## 2. Introdução

O campo do Aprendizado de Máquina tem passado por uma evolução muito rápida, graças à grande quantidade de dados disponíveis e à necessidade de modelos preditivos eficazes. Como destacam **Goodfellow et al. (2016)**, a capacidade de extrair padrões de grandes volumes de dados é o que define o sucesso das aplicações modernas de inteligência artificial.

Entre os algoritmos mais importantes, a Regressão Logística se destaca como uma técnica robusta para problemas em que precisamos decidir se algo pertence a um grupo ou não - por exemplo, se uma pessoa tem ou não uma doença cardíaca.

No entanto, o desempenho desses modelos não depende apenas de quão bem eles podem prever as coisas, mas também de quão rápido eles podem ser executados. Em linguagens de programação como o Python, quando usamos laços para processar grandes conjuntos de dados, isso pode se tornar um problema, levando muito tempo para treinar os modelos, especialmente quando os conjuntos de dados são muito grandes.

Nesse caso, conforme explicam Harris et al. (2020), surge uma técnica chamada vetorização, que usa operações matemáticas e bibliotecas otimizadas, como o NumPy, para processar vários dados ao mesmo tempo, aproveitando a capacidade da CPU e de bibliotecas como BLAS e LAPACK. Essa transição do processamento escalar para o vetorial é fundamental para a eficiência no nível do hardware, como discutido por **Patterson e Hennessy (2017)**.

Este artigo compara dois modos de implementar a Regressão Logística: o modo não-vetorizado e o vetorizado. Usando um conjunto de dados sobre doenças cardíacas do Kaggle, o estudo procura mostrar quão mais rápido é o modo vetorizado, medindo o tempo de execução e outros indicadores

importantes. O objetivo é mostrar como a escolha da forma de organizar os dados e de implementar os algoritmos pode afetar diretamente a capacidade de escalabilidade dos algoritmos de inteligência artificial.

### 3. Fundamentação teórica

#### 3.1 Regressão Logística

Modelo de classificação linear, amplamente utilizado para prever probabilidades de um dado pertencer a uma classe binária, combina simplicidade com uma base probabilística sólida. “...É possível entender a regressão logística como um complemento da regressão linear aplicada a variáveis categóricas a partir de uma função de ligação, uma generalização do teste Qui-quadrado ou, de maneira geral, um caso particular da família dos modelos lineares generalizados (GLM), que implementa uma ligação logit”, ( **Anunciação, Luís**). A regressão logística utiliza algumas funções que complementam e facilitam seu modelo, como a **Função Sigmoides**: transforma qualquer valor real em um número binário, aproximando para 0 valores muito negativos ou para 1 para valores muito positivos, formando um gráfico com formato de S; a outra função é a **Função de Custo**: guia o treinamento do modelo, mede as previsões ruins que tendem ao erro e ajusta os pesos para reduzir esse erro, repetindo essa ação até corrigir.

#### 3.2 Gradiente Descendente

Algoritmo iterativo para minimizar uma função, parte dos parâmetros e dá um “passo” na direção oposta ao gradiente, utilizando-se da taxa de aprendizado( $\alpha$ ) que faz o controle do tamanho do passo. Possui algumas variantes, como Batch(cálculo exato, caro), SGD(oscilante, rápido), Mini-batch(Equilibrado em estabilidade e velocidade). Juntamente com a função Sigmoides, é altamente recomendável utilizar do Gradiente Descendente na Regressão Logística, pois a combinação de ambas gera uma simplificação da derivada.

#### 3.3 Vetorização

Utilização de operações matriciais e vetoriais que processam todos os exemplos de uma só vez, substituindo os laços de repetição explícitos. Essas operações retiradas de bibliotecas como NumPy, executam o trabalho pesado usando rotinas otimizadas(BLAS, LAPACK, SIMD) em vez de calcular amostra por amostra. A vetorização é ligeiramente mais rápida que a não vetorizada, eliminando quase completamente os laços de repetição, sem afetar acurácia, obtendo no final o mesmo modelo, mas em um tempo consideravelmente menor.

## 4. Metodologia

Nosso banco de dados utilizados para tais experimentos foi o “Heart Disease Dataset”(Conjunto de dados de doenças cardíacas), retirado do Kaggle( um dos maiores sites de comunidades de cientista de dados) e publicado por David Lapp. Inicialmente se possui 1025 amostras e 14 colunas/categorias, porem em nosso código faremos uma redução para 13 categorias a fim de buscar um algoritmo melhor filtrado, categoria “Target” já diz quais daqueles pacientes possuem doença cardíaca ou não, e queremos que nosso algoritmo faça previsões sem tal informação de qual deles já possui. As nossas categorias trabalhadas são: age, sex, chest pain type(cp), resting blood pressure(trestbps), sérum cholestoral(chol), fasting blood sugar(fbs), resting electrocardiographic results(restecg), maximum heart rate achieved(thalach), exercise induced angina(exang), oldpeak = ST depression induced by exercise relative to rest(oldpeak), the slope of the peak exercise ST segment(slope), number of major vessels(ca), thal, e target (removido durante o Pré-processamento do Código).

### 4.1 Ambiente do experimento

Todo nosso projeto foi feito dentro do Google Colab, sendo Python 3 o tipo de ambiente de execução, em sua versão mais recente ate o momento(2026.4), rodando na maquina virtual do google “Google compute Engine” no acelerador de hardware alocado a CPU.

### 4.2 Importações de bibliotecas

Inicialmente, além do próprio banco de dados escolhido ser importado, outras bibliotecas importantes como pandas, numpy, time, plt (visualização de dados, machine learning, medição de tempo, criação de gráficos, respectivamente).

### 4.3 Pré-processamento de dados

Antes de iniciar qualquer tipo de implementação e de fato começar o modelo do nosso algoritmo, é necessário uma serie de tarefas a ser feita para nossos dados para transformar dados brutos e inconsistentes em dados adequados para o nosso modelo; essa serie de tarefas consiste em **Tratamento de valores ausentes**, onde ocorre a identificação e da limpeza desses valores caso tenha afim de filtrar melhor o algoritmo; **codificação das variáveis categóricas** para transformar em numéricas, utilizando-se do método Label Encoding que converte essas variáveis(é na aplicação desse método que também removemos a coluna target, deixando a tabela com 13 categorias); **Padronização dos dados**: técnica para ajustar os dados para uma escala em comum, facilitando o treinamento do algoritmo; e por final, **a divisão em conjunto de treinamento**(80 por cento da divisão) **e teste**(20 por cento da divisão) afim de garantir consistência e reprodutibilidade do modelo, utilizando também de um **Random state fixo**, que garante que naquele valor selecionado especifico, tenha sempre aquela mesma escolha dividida de amostras para treinamento e teste.

Tratamento de valores ausentes e demonstração do tamanho atual do dataframe:

```
PRE-PROCESSAMENTO DE DADOS:

# PRE-PROCESSAMENTO DE DADOS

# Verificar e tratar valores ausentes
missing_count = df.isnull().sum()
print('Valores ausentes por coluna:')
print(missing_count[missing_count > 0] if missing_count.sum() > 0 else 'Nenhum valor ausente encontrado.')

df = df.dropna()
print('\nTratamento concluído. Dimensões atuais do DataFrame:', df.shape)

Valores ausentes por coluna:
Nenhum valor ausente encontrado.

Tratamento concluído. Dimensões atuais do DataFrame: (1025, 14)
```

Codificação das variáveis categorias em numéricas utilizando label encoding, e remoção da coluna target:

```
[3]
✓ 2s # PRE-PROCESSAMENTO DE DADOS

# Codificação de variáveis categóricas
# Metodo Label Encoding utilizado

# Identificando colunas categóricas
categorical_cols = ['sex', 'cp', 'fbs', 'restecg', 'exang', 'slope', 'ca', 'thal']

# Aplicando Label Encoding
from sklearn.preprocessing import LabelEncoder
df_encoded = df.copy() # Criar uma cópia para não modificar o DataFrame original
for col in categorical_cols:
    le = LabelEncoder()
    df_encoded[col] = le.fit_transform(df_encoded[col])

print('Variáveis codificadas com sucesso usando Label Encoding.')
display(df_encoded.drop('target', axis=1).head())
```

... Variáveis codificadas com sucesso usando Label Encoding.

|   | age | sex | cp | trestbps | chol | fbs | restecg | thalach | exang | oldpeak | slope | ca | thal |
|---|-----|-----|----|----------|------|-----|---------|---------|-------|---------|-------|----|------|
| 0 | 52  | 1   | 0  | 125      | 212  | 0   | 1       | 168     | 0     | 1.0     | 2     | 2  | 3    |
| 1 | 53  | 1   | 0  | 140      | 203  | 1   | 0       | 155     | 1     | 3.1     | 0     | 0  | 3    |
| 2 | 70  | 1   | 0  | 145      | 174  | 0   | 1       | 125     | 1     | 2.6     | 0     | 0  | 3    |
| 3 | 61  | 1   | 0  | 148      | 203  | 0   | 1       | 161     | 0     | 0.0     | 2     | 1  | 3    |
| 4 | 62  | 0   | 0  | 138      | 294  | 1   | 1       | 106     | 0     | 1.9     | 1     | 3  | 2    |

## Padronização dos dados utilizando StandardScaler:

```
[4] Os # PRE-PROCESSAMENTO DE DADOS

# import do StandardScaler para padronizar os dados
from sklearn.preprocessing import StandardScaler

# Identificando colunas numericas (não codificadas)
numeric_cols = ['age', 'trestbps', 'chol', 'thalach', 'oldpeak']

# Inicializando o escalonador
scaler = StandardScaler()

# Criando uma copia para preservar os dados originais e aplicando a padronização
df_scaled = df_encoded.copy()
df_scaled[numeric_cols] = scaler.fit_transform(df_encoded[numeric_cols])

print('Padronização concluída com sucesso.')
display(df_scaled.drop('target', axis=1).head())
```

Padronização concluída com sucesso.

|   | age       | sex | cp | trestbps  | chol      | fbs | restecg | thalach   | exang | oldpeak   | slope | ca | thal |
|---|-----------|-----|----|-----------|-----------|-----|---------|-----------|-------|-----------|-------|----|------|
| 0 | -0.268437 | 1   | 0  | -0.377636 | -0.659332 | 0   | 1       | 0.821321  | 0     | -0.060888 | 2     | 2  | 3    |
| 1 | -0.158157 | 1   | 0  | 0.479107  | -0.833861 | 1   | 0       | 0.255968  | 1     | 1.727137  | 0     | 0  | 3    |
| 2 | 1.716595  | 1   | 0  | 0.764688  | -1.396233 | 0   | 1       | -1.048692 | 1     | 1.301417  | 0     | 0  | 3    |
| 3 | 0.724079  | 1   | 0  | 0.936037  | -0.833861 | 0   | 1       | 0.516900  | 0     | -0.912329 | 2     | 1  | 3    |
| 4 | 0.834359  | 0   | 0  | 0.364875  | 0.930822  | 1   | 1       | -1.874977 | 0     | 0.705408  | 1     | 3  | 2    |

## Divisão do conjunto de treinamento e teste:

```
[5] import numpy as np
# PRE-PROCESSAMENTO DE DADOS

# import do train_test_split para dividir o conjunto de dados em dois
from sklearn.model_selection import train_test_split

# Separando as features (X) e o alvo (y)
X = df_scaled.drop('target', axis=1)
y = df_scaled['target']

# Dividindo em treino (80%) e teste (20%)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print(f'Divisão concluída:')
print(f'X_train: {X_train.shape}')
print(f'X_test: {X_test.shape}')

# Converter para numpy (necessário para implementação não-vetorizada)
X_train = X_train.values
X_test = X_test.values
y_train = y_train.values
y_test = y_test.values
```

Divisão concluída:  
X\_train: (820, 13)  
X\_test: (205, 13)

#### 4.4 Implementações da Regressão Logística utilizando Vetorização e não Vetorização

É a partir dessa etapa onde utilizaremos da regressão logística e seus conceitos para modelar o nosso algoritmo, tanto vetorizado quanto não vetorizado. Ambas as implementações preparam as funções sigmoide, função de custo e gradiente descendente; também preparam os dados que vão ser utilizados, iniciam seus parâmetros ( $w$ ,  $b$ ). As diferenças são perceptíveis em qualquer um desses passos, enquanto na Não Vetorizada se utiliza de um Python mais “raiz”, a vetorizada com a biblioteca completa do numpy, utilizando suas rotinas ultra otimizadas que vai alterar significativamente o tempo de cada teste. Cada implementação no final vai mostrar a acurácia e o custo final delas dado a avaliação no conjunto de teste e treinamento.

#### 4.5 Medição de tempo e calculo do Speedup

dado as implementações da regressão logística e ambos os algoritmos concluídos, a medição de tempo então é começada, o código inicia 5 repetições primeiramente da versão vetorizada, e em cada uma dessas repetições os pesos ( $w$ ,  $b$ ) são inicializados do zero, o tempo iniciado utilizando `time.perf_counter()`, taxa de aprendizado a 0,31 e o algoritmo de gradiente descendente é executado por 1000 iterações. Na medição da versão Não Vetorizada é iniciada nas mesmas condições (1000 iterações e 5 repetições), tem a grande diferença na parte do gradiente descendente, por ter muitos laços For, torna muito mais lento a versão. No final de ambas as versões é retirado a média e desvio padrão das 5 repetições de cada, mostrando uma tabela desses resultados. Com os valores do tempo médio é possível calcular o speedup( tempo da não vetorizada dividido pelo tempo da vetorizada) afim de criar tabelas e gráficos com todos esses valores gerados pela medição do tempo.

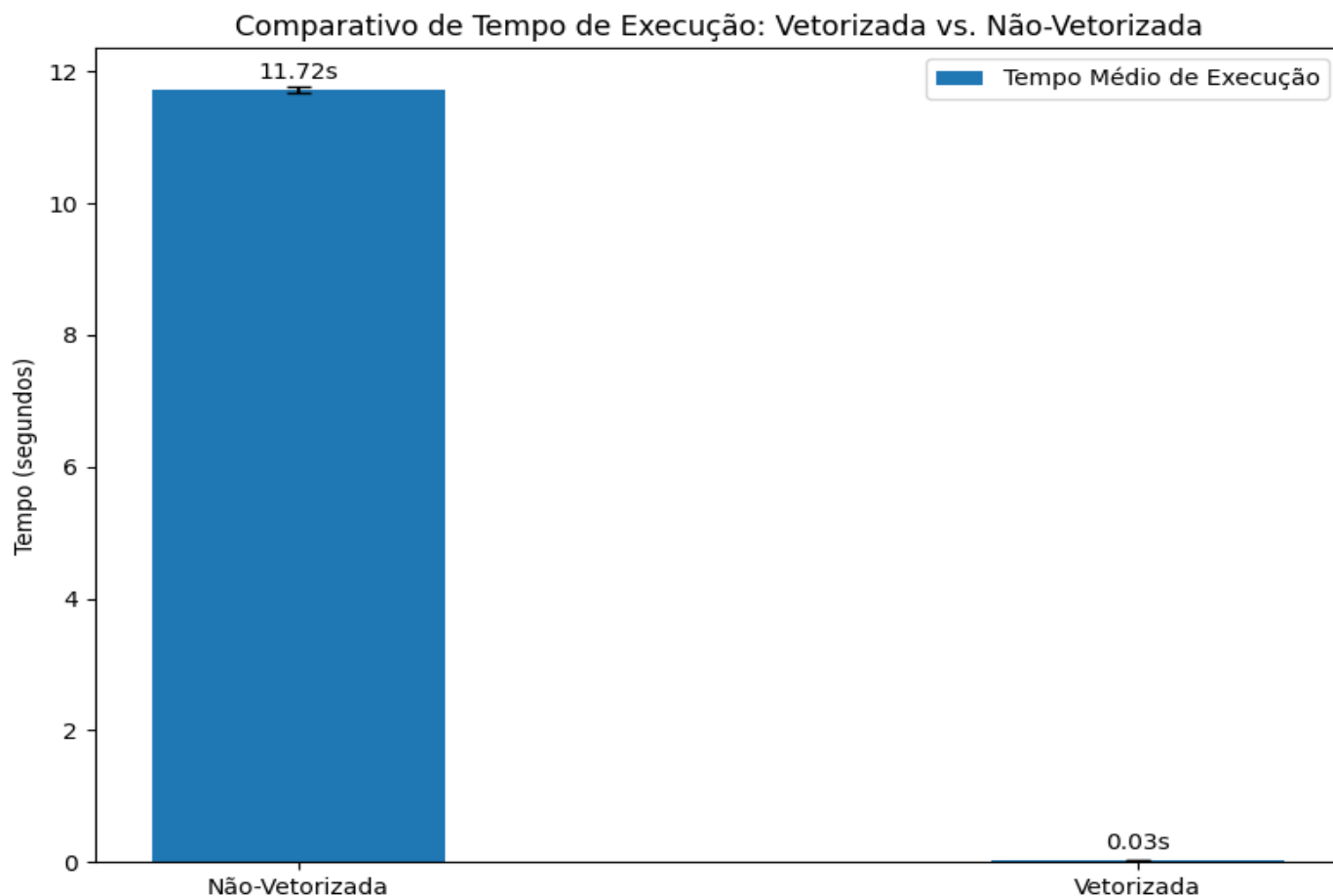
## 5. Resultados e discussão

Tabela comparativa das 5 execuções de cada implementação, suas médias e seu desvio padrão respectivamente, além do speedup.

```
Iniciando 5 execuções da VETORIZADA
V Repetição 1: 0.0287 s
V Repetição 2: 0.0281 s
V Repetição 3: 0.0283 s
V Repetição 4: 0.0282 s
V Repetição 5: 0.0297 s
Iniciando 5 execuções da NÃO VETORIZADA
NV Repetição 1: 11.7801 s
NV Repetição 2: 11.6673 s
NV Repetição 3: 11.7567 s
NV Repetição 4: 11.7083 s
NV Repetição 5: 11.6652 s

=====
TABELA COMPARATIVA
=====
MEDIA NÃO-VETORIZADA: 11.7155 s ± 0.0464
MEDIA VETORIZADA:      0.0286 s ± 0.0006
-----
SPEEDUP: 409.99x mais rápido
=====
```

Comparativo de Tempo de Execução da implementação Vetorizada e não Vetorizada.



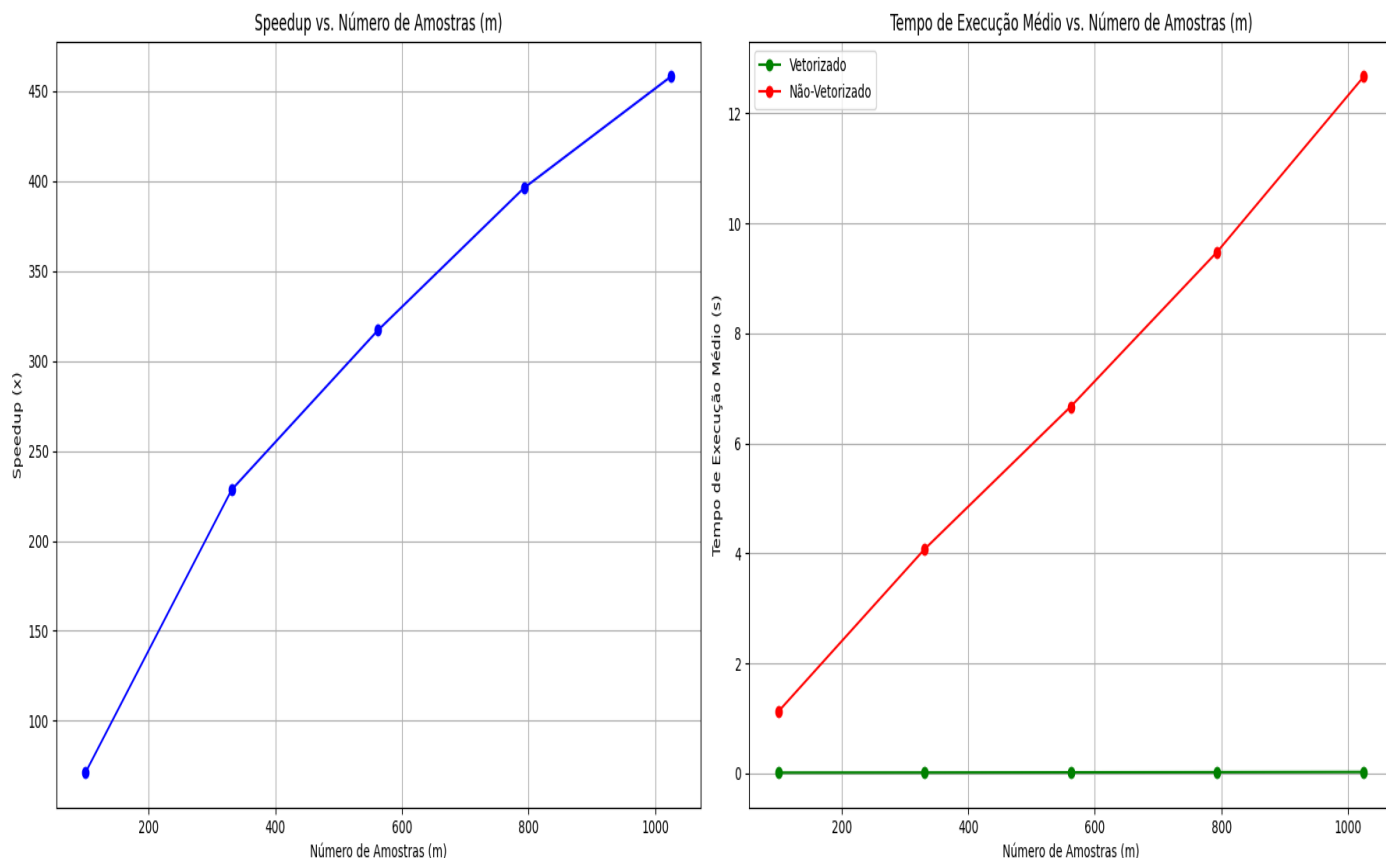


Para criar o gráfico de Speedup x numero de amostras e do tempo de execução Médio x numero de amostras, foi necessário recompilar a medição de tempo, decidimos então definir a cada uma das 5 repetições uma quantidade de amostras, cada uma maior que a outra até a ultima ser com todas, para criar gráfico de linha de maneira consistente e mostrar o tempo de cada implementação crescente pelo aumento das amostras.

Recompilação das novas amostras, média de tempo e speedup.

```
Iniciando experimento para variar o número de amostras (m)...  
  
Testando com m = 100 amostras...  
  Média de tempo vetorizado: 0.0158 s  
  Média de tempo não-vetorizado: 1.1258 s  
  Speedup para m=100: 71.18x  
  
Testando com m = 331 amostras...  
  Média de tempo vetorizado: 0.0178 s  
  Média de tempo não-vetorizado: 4.0778 s  
  Speedup para m=331: 228.47x  
  
Testando com m = 562 amostras...  
  Média de tempo vetorizado: 0.0210 s  
  Média de tempo não-vetorizado: 6.6705 s  
  Speedup para m=562: 317.08x  
  
Testando com m = 793 amostras...  
  Média de tempo vetorizado: 0.0239 s  
  Média de tempo não-vetorizado: 9.4796 s  
  Speedup para m=793: 396.26x  
  
Testando com m = 1025 amostras...  
  Média de tempo vetorizado: 0.0277 s  
  Média de tempo não-vetorizado: 12.6707 s  
  Speedup para m=1025: 458.15x  
  
Experimento concluído.
```

Gráfico com base na ultima compilação e suas amostras.



## 6. Conclusão

Conclui-se que dado a quase completa remoção de laço “for” de repetição devido a biblioteca numpy que aciona o BLAS para realizar operações necessárias e o SIMD que explora o benefício de funcionar bem para tarefas repetitivas em um conjunto de dados. A vasta otimização da vetorização ganhado com esses instrumentos reduz drasticamente o tempo de realizar buscas por um conjunto de dados, em que em um mesmo conjunto, com os mesmos pesos, com as mesmas amostras e variáveis, com a mesma taxa de aprendizado, dentre outros fatores, e ainda sim a diferença ser gritante, mostra o quão fundamental é a aplicação da vetorização nos algoritmos.

## 8.Referências Bibliográficas

**LUIS ANUNCIAÇÃO.** *Conceitos e análises estatísticas com R e JASP.*

<https://anovabr.github.io/mqt/index.html#aten%C3%A7%C3%A3o>, 2021

**BISHOP, C. M.** *Pattern Recognition and Machine Learning.* Springer, 2006.

**GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A.** *Deep Learning.* MIT Press, 2016.

**HARRIS, C. R. et al.** *Array programming with NumPy.* Nature, v. 585, p. 357-362, 2020.

**McKINNEY, W.** *Python para Análise de Dados.* Novatec Editora, 2018.

**PATTERSON, D. A.; HENNESSY, J. L.** *Arquitetura de Computadores: Uma Abordagem Quantitativa.* 6. ed. Elsevier, 2017.

### LINK BANCO DE DADOS UTILIZADO

<https://www.kaggle.com/datasets/johnsmith88/heart-disease-dataset>

### LINK CODIGO FEITO NO COLAB

<https://colab.research.google.com/drive/1LCBhkae2lzu-qsHk4XAek-xS61ClsBD5?usp=sharing>